IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 26 June 2025, accepted 14 July 2025, date of publication 16 July 2025, date of current version 24 July 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3589959

RESEARCH ARTICLE

Smart Agile Prioritization and Clustering: An AI-Driven Approach for Requirements Prioritization

AYA M. RADWAN, MANAL A. ABDEL-FATTAH, AND WAEL MOHAMED
Department of Information Systems, Faculty of Computers and Artificial Intelligence, Helwan University, Cairo 12317, Egypt
Corresponding author: Aya M. Radwan (ayaradwan.fci@gmail.com)

ABSTRACT In Agile software development, requirements prioritization plays a crucial role in ensuring that critical functionalities are delivered efficiently. Traditional prioritization methods often suffer from scalability limitations, lack of automation, and difficulty in handling dependencies. This paper proposes Smart Agile Prioritization and Clustering (SAPC), an AI-driven approach that enhances requirements prioritization by leveraging Natural Language Processing (NLP), BERT embeddings, graph-based dependency modeling, and optimization techniques. The proposed model extracts and processes textual requirements, constructs a dependency graph to model interrelations, and applies PageRank to compute requirement importance. Feature fusion and dimensionality reduction using Uniform Manifold Approximation and Projection (UMAP) facilitate clustering, while Particle Swarm Optimization (PSO) determines the optimal number of clusters for efficient backlog prioritization. The effectiveness of SAPC is evaluated using functional requirements extracted from Software Requirement Specifications (SRS), product backlogs, and customer requests, along with a benchmark dataset for validation. Various machine learning algorithms are tested, with KNN and Random Forest demonstrating the highest accuracy and lowest Mean Squared Error (MSE), outperforming traditional prioritization techniques. The results highlight the potential of AI-based methods in automating and optimizing backlog management within Agile methodologies, offering a scalable and data-driven approach to enhanced requirements prioritization.

INDEX TERMS Agile prioritization, requirements clustering, BERT embeddings, PageRank, particle swarm optimization (PSO), NLP, machine learning.

I. INTRODUCTION

Requirements Engineering (RE) is a key process in software development. It involves identifying client needs, understanding the system's development context, and managing stakeholder requirements through modeling, analysis, and prioritization. It plays a crucial role in ensuring that these requirements align with predefined specifications while accommodating evolving changes throughout the software development lifecycle [1], [2]

The associate editor coordinating the review of this manuscript and approving it for publication was Dominik Strzalka.

Requirements Engineering presents several challenges, particularly in stakeholder communication, collaboration, requirements clarification, and prioritization. Effective engagement between project teams and stakeholders is essential, yet often hindered by conflicts, misaligned expectations, and the need for continuous involvement. Moreover, defining high-quality requirements and well-structured user stories is a complex and time-consuming task. Overcoming these challenges is vital for enhancing software project success and improving development efficiency [3], [4]

In Agile software development, user stories define an application's functionalities from the users' perspective, clearly conveying the value each requirement adds.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

The Product Backlog contains all identified features, with each feature represented as a user story, ensuring that the development process aligns with user needs and business value [5], [6] The Product Owner (PO) plays a crucial role in maximizing the product's value by guiding the Development Team. Their effectiveness directly impacts the team's success, requiring strong skills and expertise. As a key figure in project planning, the PO ensures transparency, manages stakeholder expectations, and facilitates clear communication across all involved parties [7]

Requirements prioritization (RP) is a fundamental aspect of Requirements Engineering (RE) that ensures the most critical features are implemented first. As an iterative process, RP aligns with Agile's adaptive and incremental approach, enabling continuous decision-making to deliver a high-quality system within evolving constraints. Agile methodologies prioritize requirements based on stakeholder input, business value, cost, risk, and urgency, ensuring flexibility in response to changing needs. Active stakeholder collaboration is essential for refining priorities throughout development, fostering responsiveness and delivering maximum value in each sprint [8]

Several software requirements prioritization techniques, such as Pairwise Comparison, MoSCoW, and Value-Based Prioritization, have been proposed, combining both quantitative and subjective approaches. However, existing prioritization techniques face challenges such as high computational costs, complexity, lack of automation, scalability issues, ambiguity, and uncertainty [2], [9].

AI-based techniques offer solutions to these limitations. For instance, fuzzy logic helps manage uncertainty and conflicting requirements, while genetic algorithms, simulated annealing, ant colony optimization, and machine learning improve prioritization accuracy and adaptability. Integrating AI can enhance decision-making, automate processes, and optimize prioritization for dynamic and complex projects [10]

The Product Backlog (PB) is a prioritized list of tasks, features, and requirements for a development project. In AI-driven applications, some features rely on AI while others do not, leading to the concept of a Smart Product Backlog (SPB). The SPB automates backlog prioritization, reducing reliance on the Product Owner's manual decisions. This enhances efficiency, minimizes bias, aligns development with stakeholder needs, and ensures a more strategic and data-driven approach to planning [6]

Prioritization of requirements plays a significant role in Agile development, yet traditional methods often face limitations in scalability and objectivity. Despite recent AI-driven advancements in requirements prioritization, existing models often struggle with adaptability, automation, and real-time decision-making [8]. Building on these findings, this paper explores novel advancements in AI-driven prioritization techniques, evaluating their impact on Agile development efficiency and decision-making.

Machine Learning (ML) encompasses various algorithms and techniques that mimic human thinking by learning from data. It addresses software engineering challenges through supervised, unsupervised, and reinforcement learning approaches. ML improves software development through the utilization of predictive models to optimize algorithm selection and feature design. Implementing ML involves understanding the problem domain, gathering relevant data, performing preprocessing, and conducting thorough evaluations to ensure effectiveness [11]

Both machine learning and software requirements engineering are important research fields, with machine learning being used in many software engineering processes. Software requirements are categorized and prioritized using machine learning techniques such as text feature extraction and classification algorithms, leveraging vast amounts of data and domain knowledge acquired during the development process [12]

Natural Language Processing (NLP), a branch of Artificial Intelligence (AI), focuses on enabling computers to process, understand, and generate human language. It involves processing, understanding, and generating text or speech data. NLP is closely related to Machine Learning (ML), as many NLP tasks leverage ML techniques, particularly deep learning models, to learn patterns and make predictions from large text datasets. One crucial NLP application is semantic similarity analysis, which measures how closely two pieces of text (words, sentences, or documents) relate in meaning. Traditional methods relied on lexical similarity (matching words), but modern approaches use deep learning, bert embedding, and transformer models to capture semantic relationships [13]

This work proposes an AI-driven approach for prioritizing the Product Backlog and optimizing Agile sprint planning. It leverages NLP techniques for preprocessing, BERT embeddings for feature extraction, and customer ranking based on business value and urgency. K-means clustering is used to organize backlog items, while Particle Swarm Optimization refines the clustering process, ensuring more efficient sprint planning. This approach reduces manual effort, enhances prioritization accuracy, and offers a unique advantage for automating sprint planning in Agile environments.

The remainder of this paper is structured as follows. Section II provides a comprehensive literature review, discussing NLP and Machine Learning techniques in requirements prioritization. Section III describes the research methodology used in this study. Section IV presents the results of our proposed model, while Section V discusses experimental study and evaluation. Section VI outlines the study's limitations. Finally, Section VII concludes the research with key findings and future directions.

II. LITERATURE REVIEW

To identify related studies, Literature Review was conducted following the guidelines established in the AI-driven prioritization techniques study [8]. The search was performed across Scopus, IEEE Xplore, Google Scholar, and ACM Digital Library, focusing on research that applies Artificial

Intelligence (AI), Natural Language Processing (NLP), clustering, and optimization algorithms to Agile requirements prioritization. Following the SLR methodology, relevant works were reviewed and selected based on methodology, evaluation techniques, and contributions to automated prioritization. Additionally, other related works outside the SLR were considered, including studies on AI-driven backlog management, NLP-based requirements classification, and optimization techniques for Agile decision-making. These studies contribute valuable insights into task automation, dependency handling, and stakeholder collaboration, further highlighting gaps in existing prioritization models.

Izhar et al. [14] present the rule-based automated requirements prioritization RAR-P model to enhance requirements prioritization in agile software development using natural language processing and machine learning. By applying text preprocessing, TF-IDF vectorization, cosine similarity, and K-Means clustering, the model automates requirements prioritization, ensuring more objective and efficient decision-making. This approach outperforms traditional methods by improving accuracy, stakeholder alignment, and resource allocation, However, RAR-P has limitations, such as reliance on high-quality textual data, potential biases in machine learning models, and the computational complexity of clustering algorithms, which may affect scalability in large projects. Despite these challenges, it presents a promising advancement in agile project management.

Ahmed et al. [15] applied the NLP-based quality attributes extraction and prioritization (QAEP) framework that automate requirements prioritization in agile software development using regular expressions and NLP parsing to extract quality attributes and ranks them based on frequency, stakeholder impact, and criticality factor value. While it enhances automation and efficiency, limitations include dependence on predefined rules, difficulty with complex user stories. Despite this, QAEP offers a significant advancement in automated quality attribute prioritization.

Obike et al. [16] proposed the feature engineering for agile requirement management using semantic analysis approach to enhance requirements prioritization in agile software development by leveraging TF-IDF weighted Word2Vec for feature extraction, Latent Dirichlet Allocation for topic modeling, and principal component analysis for dimensionality reduction. These techniques improve semantic representation and classification of requirements, reducing ambiguity in textual data. However, the method has limitations, including reliance on predefined datasets (PROMISE and Coquina), potential bias in topic modeling, and challenges in handling highly complex requirements. Notably, requirement dependencies were not considered, which may impact prioritization accuracy in scenarios where interdependencies influence development sequences. Despite these constraints, it contributes to more adaptive and automated requirement management strategies in agile environments.

Gaona-Cuevas et al. [6] present the smart product backlog (SPB) classification model incorporates natural language processing techniques for text preprocessing and feature extraction, utilizing TF-IDF and word vector representations to enhance requirement classification. It applies logistic regression, k-nearest neighbors, and support vector machines to categorize user stories based on AI feasibility, reducing manual effort. However, limitations include dependence on predefined datasets, subjectivity in expert labeling, and scalability constraints. Additionally, it does not explicitly consider requirement dependencies or stakeholder collaboration, which are crucial for agile software development. Despite these challenges, SPB improves AI-driven backlog management and automated requirement classification.

Shafiq et al. [17] propose NLP4IP, a natural language processing NLP-based approach for issue prioritization in agile software development. The model leverages text vectorization, priority attributes, and effort estimation (story points) to rank issues dynamically, using machine learning algorithms such as XGBoost, support vector machine, and random forest. It was validated on 19 projects achieving an 81% top@3 accuracy. While NLP4IP integrates stakeholder-defined priority and effort attributes, it does not explicitly consider requirement dependencies. Additionally, subjectivity in stakeholder-assigned priorities and reliance on predefined datasets pose challenges?.

Sachdeva et al. [18] propose an NLP-based approach to prioritize user stories in agile software development. The model extracts actors, functions, and values from user stories using Stanford CoreNLP and converts them into UML activity diagrams to determine execution dependencies. Prioritization is then based on business value and process flow order. While this method improves prioritization accuracy, it depends on predefined rules, lacks automation in business value assessment, and requires manual involvement from the product owner. The approach considers requirement dependencies but does not directly facilitate stakeholder collaboration.

Sami et al. [19] propose a large language model (Llm)-based approach for prioritizing software requirements in agile software development. Their web-based tool integrates react, flask, and OpenAI's GPT models to automate user story generation and prioritization using techniques such as Analytic Hierarchy Process (AHP) and MoSCoW. The system was validated through a systematic literature review case study, demonstrating efficiency improvements in requirement prioritization. However, the approach does not explicitly consider requirement dependencies, and LLM hallucinations pose a reliability challenge. While it facilitates stakeholder collaboration via JIRA and Trello integration, the reliance on costly OpenAI services limits accessibility. Despite these challenges, the study highlights the potential of LLMs in enhancing automation and decision-making in agile requirement management?.

Bugayenko et al. [20] acknowledge several limitations in their PSO-based task prioritization model. While the

approach optimizes task ranking based on historical backlog data, it does not explicitly consider stakeholder collaboration or interactions, which are crucial in Agile decision-making. This reliance on backlog history also makes it less effective for projects with limited task data. Additionally, the method is tailored for Puzzle-Driven Development (PDD) and may require adaptation for other Agile frameworks. In terms of accuracy, evaluation using Normalized Kendall Distance (Kn) demonstrated higher precision compared to traditional prioritization methods. However, the use of linear models may limit the ability to capture complex task dependencies.

Sharma and Kumar [21] proposed an NLP-based approach for automating release planning in Agile development. Their method clusters user stories into releases using an RV coefficient algorithm, which measures similarity based on a word corpus. This approach improves dependency handling by ensuring related stories are grouped together. Validation on a banking project showed 82.35% accuracy and a 78.57%-time reduction. However, it lacks stakeholder collaboration and relies on project corpus quality, requiring manual fine-tuning.

Anand and Dinakaran [22] proposed an Apriori algorithm-based approach aimed at resolving stakeholder conflicts in Agile requirements prioritization. They achieved 89% accuracy and improved stakeholder alignment. However, the approach did not handle semantic analysis or utilize AI-driven NLP techniques.

Kumar et al. [23] proposed a K-Means-based clustering approach to group similar user stories in Agile requirement engineering. They used TF-IDF for vectorization and reported poor silhouette scores. However, they did not address semantic understanding, or stakeholder involvement.

The literature review highlights a wide range of techniques for requirements prioritization in Agile Software Development (see Table 1) However, no single standardized approach exists. Machine learning and natural language processing (NLP) methods, such as TF-IDF, Word2Vec, K-Means clustering, and classification models like SVM and XGBoost, are commonly used due to their effectiveness in semantic analysis and ranking. UML-based methods, like those in Sachdeva et al., also provide structured prioritization by mapping dependencies.

Despite these advancements, several challenges persist. Many techniques depend on predefined datasets, limiting adaptability, while others require manual stakeholder input for prioritization. Computational complexity remains a concern, particularly in clustering and deep learning models. Additionally, most ML-based approaches do not explicitly consider requirement dependencies. Notably, none of the reviewed studies utilize deep contextual embeddings like BERT, which could improve the semantic understanding of requirements, dependency handling, and prioritization accuracy.

Recently, LLM-based approaches, such as Sami et al.'s GPT-driven prioritization system, have gained attention for enhancing automation and stakeholder collaboration. However, these methods face challenges, including hallucinations and high computational costs. While ML and NLP dominate this research area, UML-based techniques offer better dependency management, and LLMs present a promising, though costly, alternative. The absence of deep contextual analysis, particularly through embeddings like BERT, highlights a research gap. Future work should explore hybrid approaches that integrate ML, NLP, and deep embeddings while addressing automation, scalability, and validation challenges.

III. RESEARCH METHODOLOGY

This research introduces Smart Agile Prioritization and Clustering (SAPC), a novel approach designed to enhance requirements prioritization and clustering in Agile methodologies. SAPC consists of three key phases. First phase: Data Handling: Requirements are gathered from software projects and evaluated by stakeholders based on business value and urgency. Second Phase: Data Preprocessing: The collected requirements undergo cleaning, normalization, and transformation using text processing techniques and feature extraction, preparing them for clustering. Third phase: Clustering & Optimization: Requirements are grouped using K-means clustering, and Particle Swarm Optimization (PSO) is applied to determine the optimal number of clusters. Finally, the results are exported, supporting decision-making in Agile sprint planning.

A. RESEARCH QUESTIONS

Despite advancements in AI-driven requirements prioritization, challenges such as dependency management, stakeholder collaboration, and optimization accuracy remain. To address these issues, this research explores the following questions:

Q1: How can AI-driven techniques, such as NLP and machine learning, improve the accuracy and efficiency of requirements prioritization in Agile development while considering dependencies and stakeholder collaboration?

Q2: How can Natural Language Processing (NLP) techniques, including BERT embeddings, enhance the extraction and prioritization of user requirements in Agile development?

Q3: How can machine learning techniques optimize requirements prioritization in Agile development by minimizing dependency conflicts and improving stakeholder collaboration, while achieving the lowest Mean Squared Error (MSE) in predictive models?

B. DATA HANDLING

This phase focuses on collecting, structuring, and ranking requirements to ensure an effective prioritization process. It consists of two main steps:

1) REQUIREMENT EXTRACTION

The dataset comprises functional requirements collected from 22 real-world software projects, with the number of requirements per project ranging from 50 to 138. To ensure

TABLE 1. Summary of the literature review.

Study	Techniques used for prioritization	Evaluation Metrics	Key Results	Dependency Consideration	Stakeholder Collaboration	Validation	Limitations
Izhar et al. [14]	NLP, ML (TF-IDF, Cosine Similarity, K-Means Clustering)	Not specified	Automates RP, improves accuracy, resource allocation, and stakeholder alignment	No	Improve stakeholder alignment	No explicit validation mentioned	Requires high-quality data, ML bias, computational complexity
Ahmed et al. [15]	Regular Expressions, NLP Parsing, Frequency Analysis, CFV	Not specified	Automates quality attribute extraction and prioritization	No	Not mentioned	No explicit validation mentioned	It depends on predefined rules, struggles with complex user stories
Obike et al. [16]	TF-IDF Weighted Word2Vec, LDA, PCA	Not specified	Enhances semantic representation, reduces ambiguity in textual data	No	Not mentioned	No explicit validation mentioned	Relies on predefined datasets, topic modeling bias, struggles with complex requirements
Gaona-Cuevas et al. [6]	NLP (TF-IDF, Word Vector Representations), Logistic Regression, k-NN, SVM	Not specified	AI-driven backlog management, reduces manual effort	No	Not considered	No explicit validation mentioned	Dataset dependency, subjectivity in labeling, scalability issues
Shafiq et al. [17]	NLP (Text Vectorization), ML (XGBoost, SVM, Random Forest), Effort Estimation	Top@3 Accuracy (81%)	Achieves 81% accuracy in issue ranking, integrates stakeholder-defined priorities	No	Uses stakeholder-defined priorities	Validated on 19 JIRA projects	Subjectivity in priorities, dataset dependency
Sachdeva et al. [18]	NLP (Stanford CoreNLP), UML Activity Diagrams, Business Value, Process Flow Order	Not specified	Improves prioritization accuracy by considering business value & execution order	Considered (via UML)	Not directly facilitated	No explicit validation mentioned	Predefined rules, lacks automation in business value, requires manual PO input
Sami et al. [19]	LLMs (GPT-3.5), React, Flask, AHP, MoSCoW	Efficiency Improvement	Automates requirements prioritization and collaboration, but LLM hallucinations exist	No	Facilitates collaboration (JIRA, Trello)	SLR Case Study (Efficiency improvement)	LLM hallucinations, costly OpenAI services
Bugayenko et al. [20]	PSO (Particle Swarm Optimization), Historical Backlog Data	Normalized Kendall Distance	Achieves high precision compared to traditional prioritization methods	No	Not considered	Evaluated with Normalized Kendall Distance	Not effective for projects with limited data, tailored for PDD, struggles with complex dependencies
Sharma & Kumar [21]	NLP-based clustering, RV Coefficient Algorithm (word corpus similarity)	Accuracy (82.35%)	Groups related user stories, improving dependency handling and reducing time by 78.57%	Considered (clusters related stories)	Not included	Banking project (82.35% accuracy, 78.57% time reduction)	Relies on corpus quality, manual fine-tuning required
Vijay Anand [22]	Apriori algorithm	89%	resolving stakeholder conflicts in Agile requirements prioritization	NO	Considered	Accuracy-based comparison	Lacks semantic processing; no NLP/AI techniques
Kumar [23]	Kmeans , TFIDF, Cosine Similarity	Silhouette Coefficient	Grouping Similar user story	NO	Not Considered	Case study with 98 stories	No semantic understanding; basic clustering; lacks stakeholder involvement

a meaningful evaluation, a subset of 12 projects was selected for testing, covering a range of project sizes and complexity levels. Smaller projects with limited data and especially those with similar requirement counts were excluded to avoid redundancy and repetition, as such projects may lack the complexity and variation needed to assess the effectiveness of our approach. The selected projects cover diverse requirements and dependencies, making them representative of real-world Agile development scenarios. These 12 projects were selected with the aim of ensuring diversity across

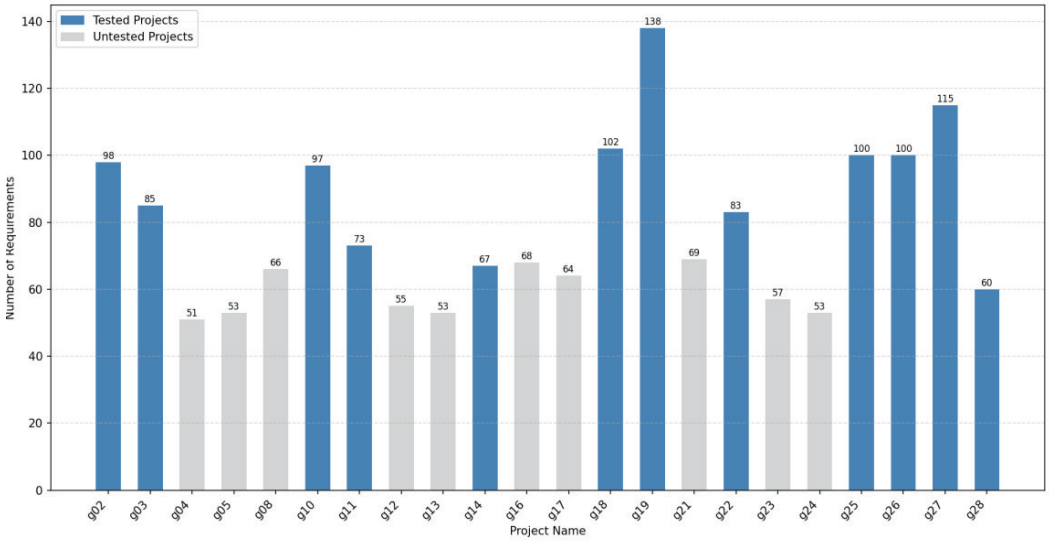

FIGURE 1. Distribution of tested and untested requirements across 22 software projects.

application domains, including public sector services, digital repositories, healthcare technology, research data management, and multimedia discovery.

The dataset is publicly available in the Mendeley Data Repository [24]. The requirements were gathered from Software Requirement Specifications (SRS), product backlogs, and customer requests, primarily focusing on system features, interactions, and development tasks, rather than explicitly specifying non-functional requirements.

Table 2 presents the application domains of the selected projects, along with their assigned project numbers for testing (PNo) and the corresponding project identifiers (PIDs) from the Mendeley repository. This diversity helps ensure that the SAPC framework is applicable to various software types and stakeholder environments, thereby supporting claims of generalizability. Figure 1 presents the tested and untested requirements across 22 software projects. The selection includes representative samples from small, medium, and large projects, with redundant small-scale projects excluded for clarity and efficiency.

TABLE 2. Domains and Descriptions of the 12 Selected Projects

P.No	P.ID	Domain
1	g02	Government Financial Reporting
2	g03	Government / Urban Planning
3	g10	Agile Training & Certification
4	g11	Government Digital Services & CMS Transformation
5	g14	Data Publishing and Management System
6	g18	Research Lab Data Management System
7	g19	Ambient Assisted Living (AAL)
8	g22	Research Data Management (RDM)
9	g25	Digital Asset Management Systems (DAMS)
10	g26	Digital Archives Management
11	g27	Institutional Repository & Research Data Management
12	g28	AI-Powered Multimedia Discovery & Citizen Science

2) REQUIREMENT STRUCTURING AND RANKING

Once extracted, the requirements are structured and ranked to facilitate prioritization. Unlike traditional expert-driven approaches, stakeholders directly rank the requirements based on business value and urgency using a scale from 1 to 5, this direct ranking approach makes the prioritization process easily reproducible by others. To ensure consistency in ranking, stakeholders are asked the following key questions for each requirement:

Q1: How critical is this requirement for achieving business goals?

Q2: How soon should this requirement be implemented?

The mean ranking score is calculated and will be combined with NLP-extracted insights in the next phase to refine prioritization. At this stage, the dataset is cleaned and prepared for further analysis, setting the foundation for preprocessing, clustering, and optimization in the subsequent phases.

To enhance prioritization accuracy, Natural Language Processing (NLP) techniques are applied to analyze requirement dependencies and relationships. The final ranking is derived by combining customer input with NLP-extracted insights, ensuring a balanced prioritization that considers both explicit user preferences and implicit textual similarities.

C. DATA PRE-PROCESSING

This phase focuses on preparing the dataset for clustering by applying text processing, feature extraction, similarity analysis, and normalization techniques. The structured and ranked requirements from Phase 1 undergo further refinement to ensure consistency and enhance prioritization.

1) TEXT PREPROCESSING

To prepare the requirements for analysis, several Natural Language Processing (NLP) techniques are applied to refine

the textual data for clustering and prioritization. The process starts with tokenization and stopword removal using NLTK, where unnecessary words are eliminated, and the text is standardized to enhance processing. Then, lemmatization is performed using spaCy to convert words to their base forms, ensuring uniform representation [25], [26]. Only content words such as nouns, verbs, adjectives, and adverbs are retained to capture meaningful semantic information. These preprocessing steps enhance the linguistic clarity of the dataset, making it more suitable for feature extraction and ranking integration. For example, the original requirement "The users are accessing their accounts through the mobile app" is transformed into "user access account mobile app" after stopword removal and lemmatization. These preprocessing steps improve the dataset's structure, making it more suitable for feature extraction and ranking integration.

2) FEATURE EXTRACTION & SIMILARITY ANALYSIS

Once the text is preprocessed, feature extraction techniques convert textual data into structured numerical representations.

Step 1: BERT Embeddings.

To capture contextual semantics, Sentence-BERT (SBERT) generates high-dimensional vector representations of requirements, preserving their meaning for clustering and prioritization. It is particularly effective in generating sentence-level embeddings while maintaining computational efficiency, making it well-suited for short textual inputs such as software requirements. The all-MiniLM-L6-v2 variant is selected for its strong balance of accuracy and speed. Recent studies have demonstrated the effectiveness of BERT-based embeddings in various NLP applications [27], with SBERT proving particularly efficient for sentence-level similarity tasks. Prior research further indicates that SBERT outperforms full BERT models in efficiency while maintaining strong performance [28], [29]. Mathematically, the embedding transformation is expressed in Equation (1), where E_{BERT} represents the BERT embeddings, $f_{BERT}(R)$ denotes the embedding function applied to requirement (R) using the Sentence-Transformer model.

$$E_{\text{BERT}} = f_{\text{BERT}}(R) = \text{SentenceTransformer('all-MiniLM-L6-v2').encode}(\ R\ ) \tag{1}$$

Step 2: Dependency-Based Feature Extraction

The second step in feature extraction leverages dependency relationships from text data, constructing a directed graph based on these dependencies and computing an importance score for each requirement using graph-based ranking. This method captures semantic relationships between requirements. The extracted features are then combined with BERT embeddings and stakeholder rankings, followed by dimensionality reduction using UMAP.

Dependency relationships between requirements are extracted using SpaCy's dependency parser, which analyzes linguistic structure [30], [31]. Each requirement undergoes syntactic analysis using dependency parsing. Key grammatical elements, such as nominal subjects (nsubj), direct objects (dobj), and root verbs (ROOT) are extracted, as they encapsulate the core meaning of the requirement. To ensure consistency and avoid case-sensitive mismatches, the extracted tokens are converted to lowercase. The extracted dependencies are stored to create a structured representation of key linguistic components. This process generates a set of dependency words for each requirement, serving as the foundation for establishing relationships between different requirements through graph-based modeling.

To maintain clarity and consistency in graph construction, only these direct syntactic dependencies were used. Indirect or ambiguous semantic constructs such as negations, conditionals, or co-references were excluded to avoid interpretational noise. A directed edge was added between requirements only if a dependency-relevant token from one appeared explicitly in the other. This structure ensures an acyclic graph suitable for PageRank analysis.

After extracting dependency relationship, a directed graph is constructed to capture relationships between requirements. Each requirement is represented as a node, and a directed edge is created if a dependency word from one requirement appears in another. This links related requirements through shared linguistic components. Using a graph structure allows for a more interpretable and structured representation of dependencies between requirements. Unlike purely numerical ranking methods, a graph-based approach models the interdependencies between requirements, ensuring that highly referenced requirements receive higher importance scores.

A graph-based ranking algorithm, specifically PageRank, is applied to determine the importance of each requirement within the dependency graph [32], [33], [34]. A requirement referenced by multiple others is considered more critical to the system. The PageRank score for each requirement is computed as shown in Equation (2), where $RP(R_i)$ is the PageRank score of requirements R_i, d is the damping factor (typically set to 0.85), InNeighbors(R_i) are the requirements linking to R_i, OutDegree(R_j) is the number of outgoing edges from R_j.

$$PR\left(R_i\right) = (1-d) + d \sum_{j \in \text{ InNeighbors }(R_i)} \frac{PR\left(R_j\right)}{\text{OutDegree}\left(R_j\right)} \tag{2}$$

This process assigns a dependency score to each requirement, representing its relative importance within the network. Once computed, the scores are normalized for consistency and then integrated with BERT embeddings and stakeholder rankings to enhance the feature representation.

To ensure semantically meaningful connections, we limited dependency extraction to direct syntactic roles specifically, nominal subjects (nsubj), direct objects (dobj), and root verbs (ROOT). These elements encapsulate the core semantic intent of each requirement. A directed edge is established from requirement B to requirement A if B explicitly contains a dependency token extracted from A. This strategy avoids

shallow lexical overlaps and results in a sparse, directed, and acyclic graph suitable for PageRank analysis. Preliminary testing indicated that incorporating additional syntactic roles, such as modifiers or negations, increased graph density and introduced semantic noise, thereby reducing clustering clarity. Consequently, the selected edge creation approach offers a more interpretable and robust structure for prioritization.

Step 3: Feature Normalization and Fusion

Before combining features, MinMaxScaler is applied to normalize values and ensure consistency across different feature scales. MinMaxScaler is chosen over standard normalization methods because it preserves the relative relationships between scores, ensuring that ranking-based features (such as business value and dependency scores) maintain their interpretability within the normalized range [35]. Unlike Z-score normalization, which standardizes values based on their deviation from the mean, MinMaxScaler maintains proportional differences between scores, making it more suitable for ranking-based data. By scaling all features to a common range, MinMaxScaler ensures that dependency scores and user rankings contribute proportionally when fused with BERT embeddings. This prevents any single feature type from dominating the final representation due to scale differences.

Once normalized, the final feature representation is created by combining three components: BERT embeddings, normalized dependency scores, and normalized mean ranks. This unified feature matrix integrates all extracted numerical representations, ensuring balanced contributions to downstream processing.

Mathematically, the feature combination can be expressed in Equation (3), where E_{BERT} represents BERT embeddings, R′ is the normalized mean rank, and D′ is the normalized dependency score.

$$F_{\text{combined}} = E_{\text{BERT}} + R' + D' \tag{3}$$

Step 4: Dimensionality Reduction (UMAP):

The Uniform Manifold Approximation and Projection (UMAP) algorithm is applied to reduce the dimensionality of the combined feature space to improve clustering efficiency. UMAP is a non-linear dimensionality reduction technique that effectively preserves both global and local structures in high-dimensional data, making it well-suited for processing complex representations such as BERT embeddings and ranking-based numerical features [36], [37]. By reducing the feature space to two dimensions (n_components=2), UMAP enables more efficient clustering and visualization while maintaining important relationships between data points. This choice was empirically validated by testing different dimensional settings, where n_components=2 consistently achieved higher Silhouette Scores and lower Davies-Bouldin Index (DBI), indicating compact and well-separated clusters. A fixed random_state=42 was set in the UMAP configuration to ensure consistent and reproducible projection results across different runs. This reduction also minimizes computational complexity, ensuring that downstream clustering algorithms operate more effectively by removing noise and redundant information. Unlike linear techniques such as PCA, UMAP is particularly beneficial for modeling non-linear dependencies in textual and semantic data, making it an ideal choice for this task. The reduced features serve as input for the subsequent clustering and prioritization processes, ensuring that requirements are effectively grouped based on both textual similarities and business-driven rankings. Mathematically, the reduced feature can be expressed in Equation (4).

$$\text{Reduced Features} = \text{UMAP}\left(F_{\text{combined}}, n_{\text{components}} = 2\right) \tag{4}$$

D. CLUSTERING AND SPRINT MAPPING

The clustering phase aims to group similar requirements based on their extracted features, ensuring that related requirements are categorized together for better prioritization. The primary goal is to prioritize the product backlog and generate sprints by clustering requirements based on their importance and dependencies. To translate these clusters into actionable Agile planning, each cluster is subsequently mapped to a sprint. This mapping relies on the assumption that requirements grouped through clustering are semantically and functionally related. The clustering process considers multiple dimensions, including textual similarity, stakeholder-assigned priority, and dependency relationships extracted from the requirement text.

K-Means clustering is used for this purpose, with the optimal number of clusters (k) determined by Particle Swarm Optimization (PSO). This approach provides an efficient and data-driven selection of k, eliminating the need for manual tuning. The following sections outline the clustering steps, the relevant mathematical equations.

1) STEP 1: PARTICLE SWARM OPTIMIZATION FOR OPTIMAL CLUSTERS

Particle Swarm Optimization (PSO) is used to automatically determine the optimal number of clusters, addressing the limitations of manual selection. PSO improves clustering accuracy by optimizing both the number of clusters and their assignments without prior knowledge [38], [39]. By dynamically adjusting swarm behavior, PSO ensures well-separated and compact clusters. PSO employs a population-based search strategy to explore multiple clustering solutions efficiently. The optimization process involves iterating through different values of k (number of clusters), applying K-Means clustering, and evaluating the clustering performance. PSO explores the search space by adjusting the positions and velocities of particles, where each particle represents a candidate value of k. The algorithm is initialized with 20 particles, each representing a candidate requirement clustering configuration. It optimizes requirement clustering through 50 iterations, ensuring well-structured assignments for sprint planning while maintaining a balance between exploration and convergence speed. During the optimization

process, multiple parameter configurations were tested to determine the best clustering results. The evaluation was based on internal clustering quality metrics, including the Silhouette Score and Davies-Bouldin Index (DBI). The final parameter values were selected based on experimental results showing the best clustering performance, with a focus on achieving well-separated and compact clusters to support effective sprint grouping. To ensure effective exploration of the solution space, we set the PSO parameters as follows: swarmsize = 20, maxiter = 50, omega = 0.7, phip = 2.0, and phig = 2.5. These values were selected empirically based on multiple test runs. The chosen configuration consistently produced well-separated and compact clusters, as evidenced by improved Silhouette Scores and lower Davies-Bouldin Index values. This setup ensures convergence stability while balancing exploration and exploitation during optimization.

2) STEP 2: K-MEANS CLUSTERING

After determining the optimal number of clusters using PSO, K-Means clustering is applied to group the data points based on their feature similarities. The K-Means++ initialization method is used to ensure optimal centroid placement, improving convergence and preventing poor clustering. Each data point is assigned to the nearest centroid, with the objective of minimizing the sum of squared distances (SSE) between points and their assigned cluster centers as provided in the equation. This iterative process updates cluster centroids until convergence, ensuring that data points within the same cluster are as similar as possible while maintaining clear separation between different clusters. Finally, the assigned cluster labels are stored, providing structured groupings of the data for further analysis, as expressed in Equation (5), where N is the total number of data points, k is the number of clusters determined by PSO, x_i is a data point, C_j is the centroid of cluster j, I_{ij} is an indicator function, which is 1 if 1 $_i$ belongs to cluster j, and 0 otherwise.

$$SSE = \sum_{i=1}^{N} \sum_{j=1}^{k} I_{ij} \left\| x_i - C_j \right\|^2 \tag{5}$$

Once the requirements are clustered, they are automatically assigned to sprints based on their cluster number and priority. Sprint assignments follow a structured mapping, where Cluster 1 is assigned to Sprint 1, Cluster 2 to Sprint 2, and so on. This ensures a systematic distribution of requirements while prioritizing high-importance tasks early in the Agile workflow. Each requirement is treated as an individual data point represented by its unified feature vector. The assigned cluster ID serves as a classification label, allowing machine learning models to evaluate how well the clustering aligns with intended sprint groupings. This step provides an additional verification layer, ensuring that the sprint assignments reflect the intended prioritization strategy. Finally, the sprint assignments are exported to a CSV file for further analysis and integration into project management tools.

E. THE TRAINING PROCESS

The training process evaluates classification performance on the clustered data to assess the effectiveness of the feature representation. The dataset is split into training and testing sets, with 40% of the data reserved for testing, ensuring a balanced distribution of cluster labels using stratified sampling. To assess classification performance, six classifiers are employed: K-Nearest Neighbors (KNN), XGBoost (XGB), Random Forest (RF), Support Vector Machine (SVM), Logistic Regression (LR), and Decision Tree (DT) [40], [41], [42], [43].

KNN classifies data points based on the majority class of their k = 7 nearest neighbors, making it effective for handling local structures. XGBoost, a gradient boosting algorithm, is optimized with 50 estimators, a learning rate of 0.05, a max depth of 4, and L2 regularization ($\lambda = 0.1$) to enhance generalization. Random Forest constructs 100 decision trees, reducing variance and improving robustness. SVM employs the Radial Basis Function (RBF) kernel with probability estimation enabled to capture non-linear relationships. Logistic Regression, a probabilistic linear classifier, is trained with maximum iterations set to 1000 to ensure convergence. Finally, Decision Tree classifies data based on recursive feature splits, using default hyperparameters to serve as a baseline model. Each classifier is trained on the training set (60%) and tested on the test set (40%), and the classification results are exported to a CSV file for further analysis, ensuring that the learned representations align with the identified clusters.

F. THE VALIDATION PROCESS

To assess the quality of clustering, the Silhouette Score and Davies-Bouldin Index (DBI) are computed. The Silhouette Score measures how well-separated clusters are, ranging from −1 (poor clustering) to 1 (strong clustering), with higher values indicating better-defined clusters. The Davies-Bouldin Index (DBI) evaluates both cluster compactness and separation, where lower values indicate higher clustering quality. These metrics ensure that the requirements are optimally grouped based on both textual similarities and business-driven priorities, supporting sprint planning and backlog prioritization.

After training the classifiers, model performance is evaluated using the F1-score to ensure reliability and generalizability. The F1-score, defined as the mean of Precision and Recall, is particularly effective for classification tasks with imbalanced class distributions. By considering both false positives and false negatives, it provides a balanced assessment of model effectiveness, making it a robust metric for evaluating predictive performance. Additionally, Mean Squared Error (MSE) quantifies prediction errors, where lower values indicate better model performance. To further assess classification effectiveness, top@k accuracy is computed, verifying whether the true label appears among the classifier's top@K predicted probabilities. This metric provides insights into

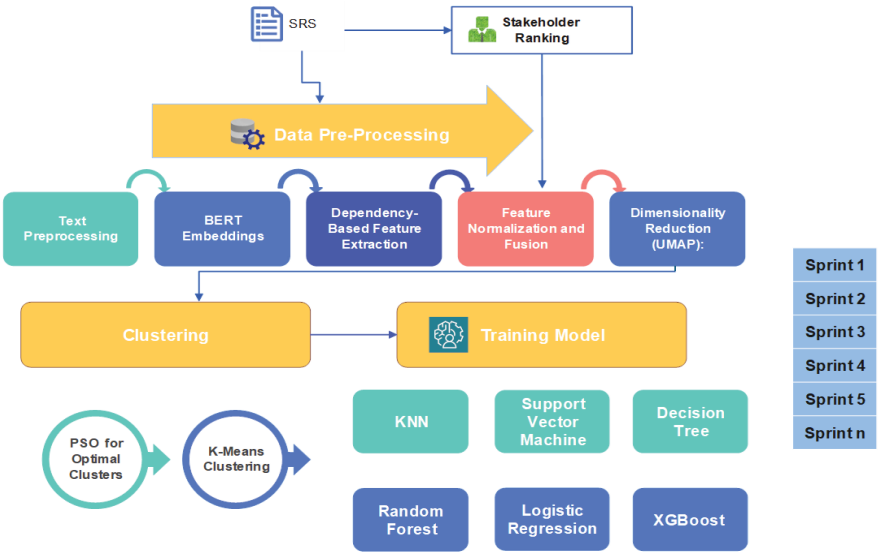

FIGURE 2. Data flow in the Smart Agile Prioritization and Clustering (SAPC) approach, illustrating the preprocessing, feature extraction, clustering, and classification stages.

model confidence and predictive strength beyond single-label classification. Higher scores across these metrics confirm improved classifier performance and alignment with the clustered data.

During the validation process, manual hyperparameter tuning was performed using 5-fold cross-validation to ensure robust classifier performance. Various configurations were tested, and those that achieved the highest average F1-score and lowest prediction error across the folds were selected.

This approach helped mitigate overfitting and ensured that each classifier was optimally tuned for the clustered requirement data.

G. MODEL ARCHITECTURE

1) OVERVIEW

The Smart Agile Prioritization and Clustering (SAPC) architecture follows a sequential, modular pipeline, as shown in Figure. 2. First, stakeholder ranking scores are collected to represent business value and urgency. In parallel, software requirements are preprocessed using standard NLP techniques, including tokenization, lemmatization, and stopword removal. From this input, two types of textual features are extracted: semantic embeddings using Sentence-BERT (SBERT) and structural features based on dependency relationships between words. The three resulting components (SBERT embeddings, stakeholder scores, and dependency scores) are normalized using MinMaxScaler to ensure uniform contribution. These features are fused into a unified feature matrix and reduced in dimensionality using Uniform Manifold Approximation and Projection (UMAP). The reduced feature set is clustered using K-Means, with the optimal number of clusters automatically determined by Particle Swarm Optimization (PSO). To evaluate prioritization performance, six classification models including K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Logistic Regression, Decision Tree, Random Forest, and XGBoost are applied to measure predictive accuracy.

A complete summary of the parameter configurations used throughout the SAPC pipeline is provided in Table 7, in Appendix A.

2) ILLUSTRATIVE EXAMPLE: SAPC END-TO-END PIPELINE (CSP PROJECT)

To illustrate how SAPC processes and prioritizes requirements, we present a representative subset of five requirements from the Citizen Services Platform (CSP) project, which is internally labeled as Project g03 in the Mendeley repository and corresponds to Project 2 in our testing set. This project falls under the domain of Government and Urban Planning. In table 3, each requirement includes metadata such as business value and urgency ranked by domain experts.

TABLE 3. Illustrative Example of Five Requirements from CSP Project with Associated Metadata and Cluster Assignment

R.ID	Requirement Summary (Truncated)	Business Value	urgency	Cluster
1	Search for Information	5	2	1
2	Submit Application	5	4	3
3	Submit Supporting Documentation	3	4	3
4	Pay Fee	5	4	3
5	Perform Completeness Review	2	1	5

Following preprocessing including tokenization, lemmatization, and stopword removal. For example, the original requirement: *"As an Applicant, I want to Submit Application, so that I can provide my information, plans and/or documents to initiate a transaction with the County."* is transformed into: *"applicant submit application provide information plan document initiate transaction county"*. Semantic embeddings are generated using Sentence-BERT. Simultaneously, spaCy's

parser extracts key dependency roles (nsubj, dobj, ROOT) to identify inter-requirement relationships. For instance:

Requirement A: "Submit Application" the dependency tokens are ['submit', 'application']. Requirement B: "Modify Application" creates an edge from B to A due to the shared token "application".

The normalized PageRank score reflects each requirement's influence in the dependency graph. This dependency example is representative and may involve requirements beyond the five illustrated.

All feature vectors (semantic, rank-based, and dependency scores) are normalized and merged. UMAP reduces dimensionality, and PSO determines the optimal number of clusters. In this case, R1 maps to Cluster 1; R2, R3, R4 to Cluster 3; and R5 ("Perform Completeness Review") to Cluster 5.

This subset demonstrates how SAPC processes real-world requirements into structured clusters based on semantic, ranking, and dependency features. The corresponding clustering and validation results (Silhouette Score, DBI, F1-score, etc.) are presented in Section IV.

IV. RESULTS

The evaluation of the proposed approach was conducted based on two key aspects: clustering quality and classification model performance. Clustering quality was assessed using two widely recognized metrics: Silhouette Score and Davies-Bouldin Index (DBI).

The Silhouette Score ranged from 0.47 to 0.7, indicating that the clustering approach achieved moderate to high cohesion and separation. A higher Silhouette Score (closer to 0.7) suggests well-defined and compact clusters, while a lower score (closer to 0.47) reflects semantic similarities and ranking variations that introduce soft boundaries between certain requirements. Similarly, the DBI values ranged from 0.4 to 0.78, further validating the clustering quality. Lower DBI values (0.4) indicate highly compact and well-separated clusters, whereas higher values (0.78) reflect some degree of overlap in ranking-based priority relationships, which may influence cluster structure. Given that text clustering operates in high-dimensional spaces with both ranking and semantic factors, achieving a DBI score below 1 confirms that the proposed clustering method effectively maintains meaningful groupings while preserving stakeholder prioritization.

The observed variability in Silhouette Score and DBI can be attributed to the combined effect of textual embeddings and stakeholder-driven ranking factors, which directly influence how requirements are grouped. The interplay between semantic relationships among requirements and ranking-based constraints contributes to differences in cluster compactness and separation across different projects. The classification performance was evaluated using six machine learning classifiers as shown at Table 4: K-Nearest Neighbors (KNN), Random Forest (RF), Logistic Regression (LR), Support Vector Machine (SVM), Decision Tree (DT),and XGBoost. Performance was assessed based on F1 Score, Mean Squared Error (MSE), and Top@3 Accuracy to measure both predictive accuracy and ranking effectiveness.

Among the classifiers, KNN achieved the highest overall performance, with the highest F1 Score (0.96) and perfect Top@3 Accuracy (1.0), indicating its strong ability to classify requirements with high precision while ensuring the correct labels consistently appeared in the top three predictions. Random Forest followed closely, achieving an F1 Score of 0.948 and the lowest MSE (0.23), demonstrating its strong generalization capabilities. The stability of Random Forest confirms the advantage of ensemble-based methods in improving predictive accuracy while reducing the risk of overfitting. Logistic Regression ranked third, maintaining a strong F1 Score (0.94) and high Top@3 Accuracy (0.99); however, its higher MSE (0.54) suggests that its linear nature may limit its ability to capture complex feature interactions.

SVM performed well in ranking tasks, achieving a perfect Top@3 Accuracy (1.000), but exhibited the highest MSE (0.686), indicating sensitivity to variations in feature distribution. Decision Tree, while maintaining a competitive F1 Score (0.919), exhibited a lower Top@3 Accuracy (0.963) and an MSE of 0.580. XGBoost underperformed compared to the other classifiers but was not significantly weaker. These results highlight that KNN and Random Forest were the most effective classifiers, excelling in both accuracy and ranking tasks. The strong performance of KNN is attributed to its ability to capture local structure within the fused feature space, which combines semantic, structural, and stakeholder information. As a distance-based classifier, KNN benefits from the contextual clarity provided by Sentence-BERT and dependency features. Similarly, Random Forest achieves robust results due to its ensemble nature, effectively handling non-linear feature interactions and reducing variance, which enhances generalization across heterogeneous projects.

The corresponding heatmap visualization of these average results is presented in figure 3. It provides a clear overview of classifier performance across all evaluation metrics.

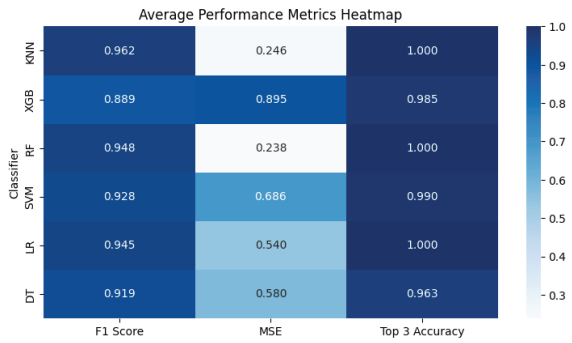

FIGURE 3. Heatmap visualization of the average classification results across twelve projects, showing F1 Score, Mean Squared Error (MSE), and Top 3 Accuracy for each model.

A summary of classification performance is provided below. Table 5 presents a comparison between the average F1-scores and the corresponding 5-fold cross-validation

TABLE 4. Classification performance of machine learning models

P. NO	K-Nearest Neighbors			XGBoost			Random Forest			Support Vector Machine			Logistic Regression			Decision Tree		
	F1 S	MSE	Top 3	F1 S	MSE	Top 3	F1 S	MSE	Top 3	F1 S	MSE	Top 3	F1 S	MSE	Top 3	F1 S	MSE	Top 3
1	0.97	0.03	1.00	0.95	0.05	1	0.95	0.05	1.00	0.95	0.05	1.00	0.95	0.05	1.00	0.95	0.05	0.95
2	0.86	1.50	1.00	0.78	2.71	0.96	0.91	0.71	1.00	0.83	2.88	1.00	0.83	2.38	1.00	0.83	2.88	0.88
3	0.97	0.03	1.00	0.97	0.03	1.00	0.97	0.03	1.00	0.97	0.03	1.00	0.97	0.03	1.00	0.97	0.03	1.00
4	0.93	0.07	1.00	0.93	0.07	1.00	1.00	0.00	1.00	0.97	0.03	1.00	0.93	0.07	1.00	1.00	0.00	1.00
5	1.00	0.00	1.00	0.78	0.22	1.00	0.89	0.22	1.00	0.92	0.07	1.00	0.96	0.04	1.00	0.89	0.22	1.00
6	0.98	0.02	1.00	0.85	0.22	1.00	0.93	0.15	1.00	1.00	0.00	1.00	1.00	0.00	1.00	0.90	0.17	1.00
7	1.00	0.00	1.00	0.93	0.18	0.96	0.96	0.04	0.96	0.96	0.04	1.00	0.96	0.04	1.00	0.96	0.04	0.96
8	0.93	0.67	1.00	0.78	2.23	0.97	0.90	0.73	1.00	0.60	4.80	1.00	0.90	0.97	1.00	0.86	1.27	0.87
9	0.93	0.60	1.00	0.70	5.03	0.93	0.90	0.90	1.00	0.97	0.30	1.00	0.87	2.87	0.97	0.84	2.13	0.90
10	1.00	0.00	1.00	1.00	0.00	1.00	0.97	0.03	1.00	1.00	0.00	1.00	1.00	0.00	1.00	0.93	0.07	1.00
11	0.97	0.03	1.00	1.00	0.00	1.00	1.00	0.00	1.00	0.97	0.03	1.00	0.97	0.03	1.00	0.90	0.10	1.00
12	1.00	0.00	1.00	1.00	0.00	1.00	1.00	0.00	1.00	1.00	0.00	1.00	1.00	0.00	1.00	1.00	0.00	1.00

TABLE 5. Comparison of F1-score and 5-fold cross-validation score.

Classifier	F1 Score	Cross-Validation Score
K-Nearest Neighbors	0.96	0.97
Random Forest	0.94	0.95
Logistic Regression	0.94	0.96
Support Vector Machine	0.92	0.96
Decision Tree	0.91	0.94
XGBoost	0.88	0.91

scores for all classifiers. Figure 4 illustrates this comparison in the form of a bar chart. The comparison highlights the consistency between evaluation metrics. Each classifier was evaluated using 5-fold cross-validation, where the dataset was split into five subsets; in each iteration, one subset served as validation and the remaining four were used for training. The final scores represent the average across folds to mitigate the risk of overfitting.

As demonstrated in Section III-E, the selected project (Project 2), formally referred to as the Citizen Services Platform (CSP) illustrates the effectiveness of SAPC across all stages. The complete clustering and classification performance for CSP is shown in Figure 5 and Figure 6. The figures show the UMAP-based clustering and the corresponding confusion matrix.

CSP project comprises 85 requirements grouped into seven clusters. The clustering, optimized using Particle Swarm Optimization (PSO), achieved an F1-score of 0.86. The UMAP visualization demonstrates well-separated clusters, reflecting the strength of the semantic and dependency-based feature representation. The confusion matrix confirms the classification performance, with a few misclassified instances across similar clusters.

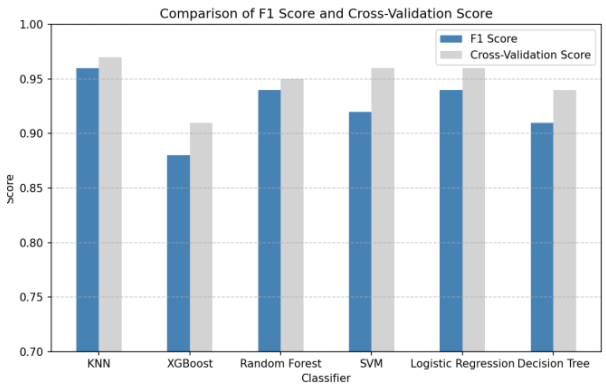

FIGURE 4. Grouped bar chart comparing F1-score and cross-validation score.

While CSP project resulted in seven clusters, this number was automatically determined using Particle Swarm Optimization (PSO) based on feature similarity and structural dependencies. The number of clusters varies across projects and reflects the natural grouping of requirements, which can correspond to sprint iterations or prioritized feature batches.

The current analysis extends CSP project by presenting an ablation study focused on evaluating the individual impact of core components including SAPC, PSO, and TF-IDF embeddings. Table 6 presents an ablation study comparing the full SAPC pipeline with two baseline configurations: (1) TF-IDF with KMeans, and (2) TF-IDF combined with PSO and dependency features.

This comparison isolates the individual contribution of key SAPC components. The TF-IDF + KMeans baseline excludes both semantic embeddings and dependency modeling, while the TF-IDF + PSO + Dependency setting removes

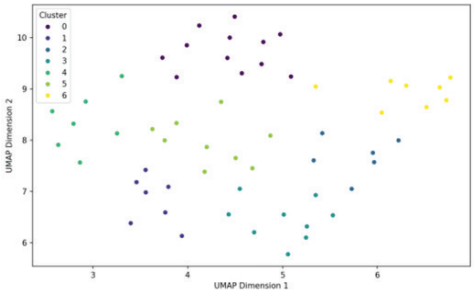

FIGURE 5. UMAP-based clustering for CSP Project.

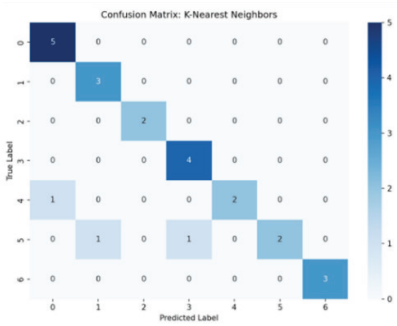

FIGURE 6. KNN confusion matrix for CSP Project.

TABLE 6. Ablation Study Comparing SAPC and TF-IDF-Based Clustering (Results Based on CSP PROJECT).

Metric	SAPC	TF-IDF+PSO+ Dependency	TF-IDF +KMeans
Silhouette Score	0.47	0.062	0.023
DBI	0.68	2.54	4.16
F1 Score	0.86	0.43	0.69
MSE	1.50	7.79	1.89
Top-3 Acc	1.00	0.87	0.97
Cross-Valid. Score	0.91	0.63	0.83

BERT embeddings but retains structural optimization. These configurations clarify the distinct impact of semantic and structural features within the pipeline.

The ablation study results presented in Table 6 demonstrate the significance of each component within the SAPC pipeline. The SAPC configuration achieved the highest overall performance, with a Silhouette Score of 0.47 and a Davies-Bouldin Index (DBI) of 0.68 indicating well-separated and compact clusters. In contrast, the TF-IDF + KMeans baseline yielded poor clustering cohesion (Silhouette Score = 0.023, DBI = 4.16), as values below 0.25 typically reflect poor cohesion. Classification performance also improved with SAPC, reflected in a higher F1-score (0.86 vs. 0.69), lower MSE (1.50 vs. 1.89), and perfect Top-3 Accuracy. These results demonstrate that removing semantic embeddings or optimization components significantly degrades both clustering and classification quality.

The results indicate that while traditional approaches perform reasonably, they lack the consistency offered by SAPC. The study confirms that each SAPC module semantic embeddings, dependency modeling, UMAP, and PSO meaningfully enhances system performance and is essential for reliable Agile sprint planning. This consistency stems from SAPC's ability to leverage machine learning outputs, enabling evaluation through quantitative metrics such as F1-score and MSE, unlike traditional methods such as MoSCoW, which rely on qualitative rankings and lack formal validation mechanisms.

To ensure reproducibility, the full execution environment specifications and Python library configurations used in the SAPC pipeline are provided in Appendix B.

V. DISCUSSION

The SAPC approach addresses several key limitations of existing requirements prioritization techniques. Many traditional methods rely on manual stakeholder input, which can introduce subjectivity and limit scalability. SAPC reduces this issue by combining stakeholder input with automated dependency modeling using PageRank, resulting in a more objective and consistent prioritization process. Unlike earlier techniques that use shallow text representations such as TF-IDF, SAPC applies Sentence-BERT embeddings to capture deeper semantic meaning. This improves the clustering and classification of requirements. In addition, SAPC uses Particle Swarm Optimization (PSO) to automatically determine the optimal number of clusters, removing the need for manual tuning. The approach also includes a comprehensive evaluation using F1-score, Mean Squared Error (MSE), Silhouette Score, and Davies-Bouldin Index (DBI), which are often missing in previous studies.

These improvements demonstrate that SAPC introduces a novel approach to Agile requirements prioritization by integrating automation, semantic analysis, and performance evaluation. The approach is distinguished by its fusion of deep semantic understanding, structural dependency modeling, and stakeholder input, offering a level of integration not typically achieved in prior AI-based prioritization frameworks.

The preprocessing step played a crucial role in improving model performance. Unlike previous studies such as NLP4IP, which relied on TF-IDF embeddings, our approach leveraged BERT embeddings, significantly enhancing feature representation. TF-IDF captures only word frequency, failing to understand semantic relationships, leading to higher Mean Squared Error (MSE) and poor cluster quality [17]. This limitation caused models to struggle with context-dependent classifications, increasing misclassification rates.

Similarly, previous approaches such as the RAR-P model [14] relied solely on shallow TF-IDF representations, which limited their capacity to capture contextual semantics. In contrast, the SAPC framework employs Sentence-BERT embeddings, offering deeper semantic awareness that enhances both prioritization accuracy and clustering quality.

Before BERT, when TF-IDF was used in preprocessing, the Silhouette Score was only 0.07, indicating weak clustering cohesion and poor separation. This suggests that

TF-IDF failed to capture meaningful relationships between requirements, leading to overlapping and inconsistent clusters. With BERT embeddings, the Silhouette Score improved significantly, ranging from 0.47 to 0.7, confirming that deep contextual representations enhance cluster quality, making classification more effective.

BERT embeddings further enabled classifiers like KNN and Random Forest to achieve higher accuracy by providing context-aware numerical representations of requirements. This is reflected in the lower MSE and higher F1 scores across all models, demonstrating that semantic-rich feature representations reduce ambiguity in requirement classification.

Additionally, dependency graph-based feature extraction complemented BERT embeddings by preserving structural relationships between words. This ensured that related requirements were more accurately clustered, minimizing inconsistencies in the feature space and enhancing classification performance.

SAPC approach automates both requirement prioritization and clustering by integrating BERT embeddings, stakeholder ranking, and dependency modeling, with performance validated using quantitative metrics such as MSE and F1-score. In contrast, [19] uses LLMs primarily for content generation, while prioritization is performed manually through frameworks like AHP and MoSCoW, with no structured validation or optimization process. This highlights the advantage of SAPC approach in delivering reproducible, data-driven results tailored to Agile planning needs.

Overall, shifting from TF-IDF to BERT transformed both clustering and classification, making our approach more robust than NLP4IP while considering dependency and stakeholder collaboration, which are critical for achieving effective requirement prioritization in Agile methodologies. This highlights the importance of deep learning-based embeddings in requirement classification tasks, particularly when handling complex textual data within Agile frameworks.

Unlike existing embedded prioritization frameworks, SAPC uniquely combines semantic modeling, dependency analysis, stakeholder input and dimensionality reduction using UMAP in a unified automated pipeline, addressing several key limitations in prior approaches.

VI. LIMITATIONS

Despite the effectiveness of the Smart Agile Prioritization and Clustering (SAPC) approach in automating and optimizing requirements prioritization, several limitations should be acknowledged. Computational complexity remains a key challenge, as the integration of BERT embeddings, PageRank, and PSO-driven clustering introduces high computational costs, particularly in terms of runtime performance and resource consumption when processing large-scale requirement datasets.

The dependency graph construction and ranking steps demand significant memory and processing power, which may affect scalability in real-time Agile environments. Additionally, SAPC lacks real-time adaptability, as it currently operates as a batch-processing model. Agile environments are highly dynamic, and requirements evolve throughout the development lifecycle; however, SAPC does not yet support continuous learning or real-time updates. Future improvements could incorporate online learning techniques to enhance adaptability and responsiveness to changing requirements. Another challenge is model explainability and interpretability, as stakeholders may require greater transparency in how machine learning decisions are made. Ensuring that prioritization and clustering results are easily interpretable for non-technical stakeholders is crucial for fostering trust and usability. Addressing these limitations through computational optimizations, real-time adaptability enhancements, and improved interpretability mechanisms could further strengthen SAPC's applicability in Agile development, making it more scalable, adaptive, and transparent for real-world implementation.

VII. CONCLUSION

This research presents SAPC, a novel AI-driven approach for requirements prioritization and clustering in Agile development. By integrating BERT embeddings, dependency graph modeling, and PSO-driven clustering, SAPC effectively automates prioritization while considering requirement dependencies and stakeholder inputs. The results indicate that KNN and Random Forest consistently deliver superior performance, reducing MSE and improving ranking accuracy compared to traditional methods.

The findings emphasize the importance of deep contextual embeddings and graph-based ranking in improving Agile backlog management. Unlike existing approaches that rely on predefined datasets or manual ranking, SAPC leverages machine learning to dynamically adjust prioritization based on real-world dependencies and stakeholder preferences.

Future work will include extending the evaluation across different domains, such as safety-critical and embedded systems, as well as large-scale open-source repositories like JIRA, to assess the generalizability and scalability of the proposed model beyond general-purpose software projects. Research should also focus on enhancing computational efficiency, real-time adaptability, and model interpretability by incorporating explainable AI (XAI) techniques, such as visual dashboards, to further optimize the approach's scalability and usability in Agile environments. Additionally, exploring hybrid models that combine large language models (LLMs) with traditional machine learning could improve prioritization accuracy while reducing processing overhead. Extending SAPC to support real-time sprint planning and adaptive backlog adjustments would further strengthen Agile decision-making. Future enhancements to the SAPC framework may explore the integration of deep learning architecture such as Graph Neural Networks (GNNs) or fine-tuned transformer models to further improve prioritization and clustering quality particularly when applied to larger datasets.

While the current model incorporates stakeholder input for urgency and business value, it is designed to dynamically

TABLE 7. Parameter Configuration Summary for the SAPC Pipeline.

Component	Parameter	Value / Description
Prioritization	Stakeholder Input	Mean of Business Value and Urgency (scale 1–5)
	Semantic Embedding	Sentence-BERT (all-MiniLM-L6-v2)
	Dependency Score	PageRank over dependency graph
	Normalization	MinMaxScaler
Feature Construction	Feature Vector	SBERT + Dependency + Mean Rank
Dimensionality Reduction	UMAP	n_components = 2, random_state = 42
Clustering	PSO Parameters	swarmsize = 20, maxiter = 50, omega = 0.7, phip = 2.0, phig = 2.5
	K-Means	init = k-means++, n_init = 20
Classification	KNN	k = 7
	SVM	RBF kernel, probability enabled
	Logistic Regression	max_iter = 1000
	Random Forest	n_estimators = 100
	XGBoost	n_estimators = 50, learning_rate = 0.05, max_depth = 4, $\lambda = 0.1$
	Decision Tree	Default settings

TABLE 8. Summary of python libraries used and their purposes.

Library	Purpose
sentence-transformers	SBERT embeddings for semantic similarity
spaCy	Lemmatization and dependency parsing
scikit-learn	Clustering, classification, preprocessing, metrics
pandas	Data manipulation and handling
numpy	Numerical operations and feature processing
networkx	Building dependency graph and PageRank calculation
UMAP-learn	Dimensionality reduction
pyswarm	Particle Swarm Optimization (PSO)
matplotlib, seaborn	Visualizations and plotting

accept additional attributes. Future work will explore integrating effort or time estimation to enhance sprint workload balancing and improve planning accuracy.

By advancing AI-driven prioritization, this study contributes to more efficient, scalable, and automated backlog management, showing the way for intelligent Agile software development processes.

APPENDEX A

See Table 7.

APPENDEX B

The experiments were conducted using Python 3.12 on a local machine with an Intel Core i7-13700H CPU, 16 GB RAM, and Intel Iris Xe integrated graphics (128 MB dedicated, ∼8 GB shared memory). The SAPC pipeline ran successfully without requiring dedicated GPU acceleration. The source code and implementation files are publicly available at the following GitHub repository [44], and the key Python libraries and their purposes are listed in Table 8.

ACKNOWLEDGMENT

The Author Aya M. Radwan would like to express my deepest gratitude to Prof. Manal for her invaluable guidance and unwavering support throughout my research. Her expertise and insightful feedback have been essential in shaping the direction and quality of this study. The Author Aya M. Radwan would also like to thank the anonymous reviewers for their thoughtful comments and constructive suggestions, which helped improve the quality of this work. The Author Aya M. Radwan also sincerely thankful to the Software Engineering Program for their steadfast support during my qualification year. Their encouragement and the resources they provided were crucial in overcoming obstacles and bringing this research to a successful conclusion.

REFERENCES

[1] P. A. Laplante and M. Kassab, *Requirements Engineering for Software and Systems*. New York, NY, USA: Auerbach, 2022.

[2] F. Hujainah, R. B. A. Bakar, M. A. Abdulgabber, and K. Z. Zamli, "Software requirements prioritisation: A systematic literature review on significance, stakeholders, techniques and challenges," *IEEE Access*, vol. 6, pp. 71497–71523, 2018.

[3] A. Radwan, A. Abdo, and S. A. Gaber, "An approach for requirements engineering analysis using conceptual mapping in healthcare domain," *Int. J. Adv. Comput. Sci. Appl.*, vol. 12, no. 8, pp. 1–16, 2021.

[4] A. Gupta, G. Poels, and P. Bera, "Using conceptual models in agile software development: A possible solution to requirements engineering challenges in agile projects," *IEEE Access*, vol. 10, pp. 119745–119766, 2022.

[5] C. A. Dos Santos, K. Bouchard, and F. Petrillo, "AI-driven user story generation," in *Proc. Int. Conf. Artif. Intell., Comput., Data Sci. Appl. (ACDSA)*, Feb. 2024, pp. 1–6.

[6] M. Gaona-Cuevas, V. Bucheli Guerrero, and F. H. Vera-Rivera, "The smart product backlog: A classification model of user stories," *IEEE Access*, vol. 12, pp. 150008–150019, 2024.

[7] K. Kantola, J. Vanhanen, and J. Tolvanen, "Mind the product owner: An action research project into agile release planning," *Inf. Softw. Technol.*, vol. 147, Jul. 2022, Art. no. 106900.

[8] A. M. Radwan, M. A. Abdel-Fattah, and W. Mohamed, "AI-driven prioritization techniques of requirements in agile methodologies: A systematic literature review," *Int. J. Adv. Comput. Sci. Appl.*, vol. 15, no. 9, pp. 1–16, 2024.

[9] K. S. Ahmad, N. Ahmad, H. Tahir, and S. Khan, "Fuzzy-MoSCoW: A fuzzy based Moscow method for the prioritization of software requirements," in *Proc. Int. Conf. Intell. Comput., Instrum. Control Technol. (ICICICT)*, Jul. 2017, pp. 433–437.

[10] R. Anwar and M. B. Bashir, "A systematic literature review of AI-based software requirements prioritization techniques," *IEEE Access*, vol. 11, pp. 143815–143860, 2023.

[11] S. Mirjalili, H. Faris, and I. Aljarah, "Introduction to evolutionary machine learning techniques," in *Evolutionary Machine Learning Techniques: Algorithms and Applications*. Singapore: Springer, 2020, pp. 1–7.

[12] M. D. Nagpal, K. Malik, and A. Kalia, "A comprehensive analysis of requirement engineering utilizing machine learning techniques," *Design Eng.*, vol. 2021, pp. 2662–2678, Dec. 2021.

[13] D. Chandrasekaran and V. Mago, "Evolution of semantic similarity—A survey," *ACM Comput. Surveys (CSUR)*, vol. 54, no. 2, pp. 1–37, Feb. 2021.

[14] R. Izhar, K. Cosh, and S. N. Bhatti, "Enhancing agile software development: A novel approach to automated requirements prioritization," in *Proc. 21st Int. Joint Conf. Comput. Sci. Softw. Eng. (JCSSE)*, Jun. 2024, pp. 286–293.

[15] M. Ahmed, S. U. R. Khan, and K. A. Alam, "An NLP-based quality attributes extraction and prioritization framework in agile-driven software development," *Automated Softw. Eng.*, vol. 30, no. 1, p. 7, May 2023.

[16] P. G. Obike, O. U. Obot, and V. E. Ekong, "Feature engineering for agile requirement management using semantic analysis," *J. Eng. Res. Rep.*, vol. 26, no. 9, pp. 287–304, Sep. 2024.

[17] S. Shafiq, A. Mashkoor, C. Mayr-Dorn, and A. Egyed, "NLP4IP: Natural language processing-based recommendation approach for issues prioritization," in *Proc. 47th Euromicro Conf. Softw. Eng. Adv. Appl. (SEAA)*, Sep. 2021, pp. 99–108.

[18] S. Sachdeva, A. Arya, P. Paygude, S. Chaudhary, and S. Idate, "Prioritizing user requirements for agile software development," in *Proc. Int. Conf. Adv. Commun. Comput. Technol. (ICACCT)*, Feb. 2018, pp. 495–498.

[19] M. Abdul Sami, Z. Rasheed, M. Waseem, Z. Zhang, T. Herda, and P. Abrahamsson, "Prioritizing software requirements using large language models," 2024, *arXiv:2405.01564*.

[20] Y. Bugayenko, M. Farina, A. Kruglov, W. Pedrycz, Y. Plaksin, and G. Succi, "Automatically prioritizing tasks in software development," *IEEE Access*, vol. 11, pp. 90322–90334, 2023.

[21] S. Sharma and D. Kumar, "Agile release planning using natural language processing algorithm," in *Proc. Amity Int. Conf. Artif. Intell. (AICAI)*, Feb. 2019, pp. 934–938.

[22] R. V. Anand and M. Dinakaran, "Handling stakeholder conflict by agile requirement prioritization using apriori technique," *Comput. Electr. Eng.*, vol. 61, pp. 126–136, Jul. 2017.

[23] B. Kumar, U. K. Tiwari, D. C. Dobhal, and H. S. Negi, "User story clustering using K-means algorithm in agile requirement engineering," in *Proc. Int. Conf. Comput. Intell. Sustain. Eng. Solutions (CISES)*, May 2022, pp. 1–5.

[24] F. Dalpiaz, "Requirements data sets (User stories)," *Mendeley Data*, vol. 1, p. e8, Jul. 2018. [Online]. Available: https://data.mendeley.com/datasets/7zbk8zsd8y/1

[25] R. Dumbre, P. Ankalwar, S. Bhagwat, D. Pandit, M. Bhutada, and S. Gund, "SpaCy and NLTK NLP techniques for text summarization: A comprehensive comparison," in *Proc. Int. Conf. Mach. Learn.*, Jan. 2025, pp. 55–64.

[26] S. Jugran, A. Kumar, B. S. Tyagi, and V. Anand, "Extractive automatic text summarization using SpaCy in Python & NLP," in *Proc. Int. Conf. Advance Comput. Innov. Technol. Eng. (ICACITE)*, Mar. 2021, pp. 582–585.

[27] A. Subakti, H. Murfi, and N. Hariadi, "The performance of BERT as data representation of text clustering," *J. Big Data*, vol. 9, no. 1, p. 15, Dec. 2022.

[28] J. Mutinda, W. Mwangi, and G. Okeyo, "Sentiment analysis of text reviews using lexicon-enhanced BERT embedding (LeBERT) model with convolutional neural network," *Appl. Sci.*, vol. 13, no. 3, p. 1445, Jan. 2023.

[29] Y. Chen and Z. Zhang, "A study of sentence similarity based on the all-minilm-l6-v2 model with 'Same semantics, different structure' after fine tuning," in *Proc. 2nd Int. Conf. Image Algorithms Artif. Intell.*, Jan. 2024, pp. 677–684.

[30] M. T. Anwar, D. Trisanto, A. Juniar, and F. A. Sase, "Aspect-based sentiment analysis on car reviews using SpaCy dependency parsing and VADER," *Advance Sustain. Sci. Eng. Technol.*, vol. 5, no. 1, Apr. 2023, Art. no. 0230109.

[31] W. T. Sewunetie and L. Kovács, "Exploring sentence parsing: OpenAI API-based and hybrid parser-based approaches," *IEEE Access*, vol. 12, pp. 38801–38815, 2024.

[32] C. Musto, P. Lops, M. de Gemmis, and G. Semeraro, "Context-aware graph-based recommendations exploiting personalized PageRank," *Knowl.-Based Syst.*, vol. 216, Mar. 2021, Art. no. 106806.

[33] A. E. Onile, J. Belikov, E. Petlenkov, and Y. Levron, "A comparative study on graph-based ranking algorithms for consumer-oriented demand side management," in *Proc. IEEE Madrid PowerTech*, Jun. 2021, pp. 1–6.

[34] T. X. Dinh and T.-D. Do, "Graph-based ranking techniques for improving VLSI placement," in *Proc. IEEE 15th Annu. Ubiquitous Comput., Electron. Mobile Commun. Conf. (UEMCON)*, Oct. 2024, pp. 715–719.

[35] Z. S. Priyambudi and Y. S. Nugroho, "Which algorithm is better? An implementation of normalization to predict student performance," *AIP Conf. Proc.*, vol. 2926, no. 1, 2024, Art. no. 020110.

[36] Y. Hozumi, R. Wang, C. Yin, and G.-W. Wei, "UMAP-assisted K-means clustering of large-scale SARS-CoV-2 mutation datasets," *Comput. Biol. Med.*, vol. 131, Apr. 2021, Art. no. 104264.

[37] M. Allaoui, M. L. Kherfi, and A. Cheriet, "Considerably improving clustering algorithms using UMAP dimensionality reduction technique: A comparative study," in *Proc. Int. Conf. Image Signal Process.* Cham, Switzerland: Springer, Jan. 2020, pp. 317–325.

[38] H. Amdouni, G. Manita, D. Oliva, E. H. Houssein, O. Korbaa, and S. Zapotecas-Martínez, "Dynamic social particle swarm optimization for automatic clustering," *Proc. Comput. Sci.*, vol. 246, pp. 1409–1418, Jan. 2024.

[39] G.-B. Chen, A. Song, C.-J. Zhang, X.-F. Liu, W.-N. Chen, Z.-H. Zhan, J.-H. Zhong, J. Zhang, and X.-M. Hu, "Automatic clustering approach based on particle swarm optimization for data with arbitrary shaped clusters," in *Proc. 7th Int. Conf. Intell. Control Inf. Process. (ICICIP)*, Dec. 2016, pp. 41–48.

[40] W. Chen, S. Chen, H. Zhang, and T. Wu, "A hybrid prediction model for type 2 diabetes using K-means and decision tree," in *Proc. 8th IEEE Int. Conf. Softw. Eng. Service Sci. (ICSESS)*, Nov. 2017, pp. 386–390.

[41] B. Ullah, M. Kamran, and Y. Rui, "Predictive modeling of short-term rockburst for the stability of subsurface structures using machine learning approaches: T-SNE, K-means clustering and XGBoost," *Mathematics*, vol. 10, no. 3, p. 449, Jan. 2022.

[42] M. Aamir and S. M. Ali Zaidi, "Clustering based semi-supervised machine learning for DDoS attack classification," *J. King Saud Univ. - Comput. Inf. Sci.*, vol. 33, no. 4, pp. 436–446, May 2021.

[43] C. Zhu, C. U. Idemudia, and W. Feng, "Improved logistic regression model for diabetes prediction by integrating PCA and K-means techniques," *Informat. Med. Unlocked*, vol. 17, Aug. 2019, Art. no. 100179.

[44] A. Radwan. *GitHub Repository*. Accessed: Jun. 2025. [Online]. Available: https://github.com/ayaradwan/SAPC

AYA M. RADWAN was born in Cairo, Egypt, in 1993. She received the B.S. and M.Sc. degrees in software engineering from Helwan University, Cairo, in 2015 and 2021, respectively. She is currently pursuing the Ph.D. degree in software engineering with Helwan University. She is a Lecturer Assistant with the Faculty of Computers and Artificial Intelligence, Helwan University.

MANAL A. ABDEL-FATTAH received the Ph.D. degree in information systems from the Faculty of Computers and Information, Cairo University. She was a Business Development Consultant with the Management National Institute and the Project Manager for the Ministry of State and Administrative Development. She is currently a Professor with the Faculty of Computers and Artificial Intelligence, Helwan University. She has supervised numerous master's and Ph.D. theses. Her research interests include big data analytics, data mining, and evaluation methodologies. She is a reviewer of several information systems journals.

WAEL MOHAMED received the B.S. degree in software engineering, and the M.Sc. and Ph.D. degrees in information systems. He is currently an Assistant Professor with the Faculty of Computers and Artificial Intelligence, Helwan University. His research interests include big data analytics, data mining, and software engineering.